FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Gavryl Garlitos
Section:	AN43
Professor:	Crisola G. Tan
Date:	October 13 2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects

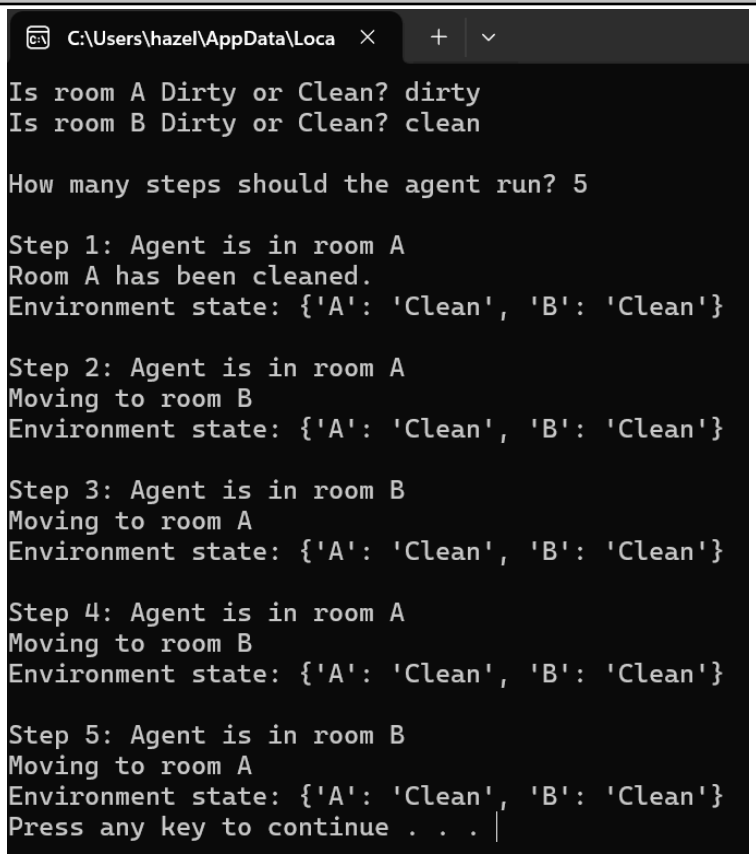
- If-Else statements
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

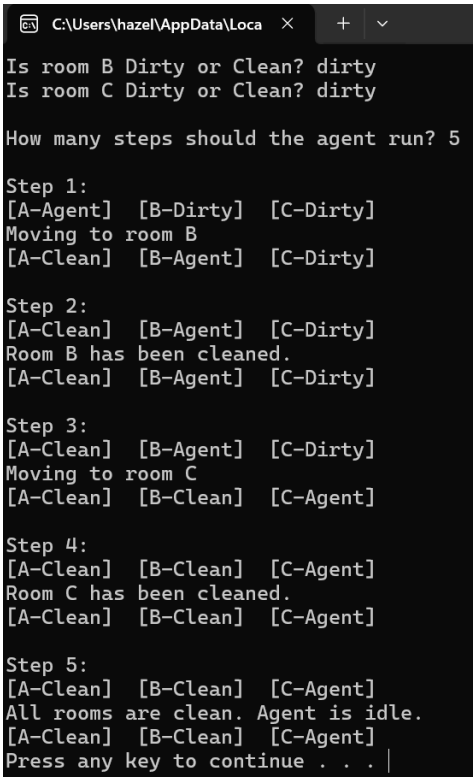
Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

Task 1: Set Up the Environment:

```
def initialize_environment():
    """Allows the user to set the initial state (dirty or clean) for Rooms A and B."""
    print("--- Set Initial State for Rooms ---")
```

```
    env = {}
    for room in ['A', 'B']:
        state = input(f"Enter state for Room {room} (dirty/clean): ").strip().lower()
        while state not in ['dirty', 'clean']:
            state = input(f"Invalid input. Enter state for Room {room} (dirty/clean): ").strip().lower()
        env[room] = state
    return env

def is_dirty(env, room):
    """Checks if a specific room is dirty."""
    return env[room] == 'dirty'

def clean_room(env, room):
    """Cleans a specific room by updating its state in the environment."""
    env[room] = 'clean'
```

Task 2: Implement the Rule-Based Agent:

```
class VacuumAgent:
    def __init__(self, start_location='A'):
        self.location = start_location

    def perceive(self, env):
        """Perceives the state of the room the agent is currently in."""
        return env[self.location]

    def act(self, env):
        """
        Decides an action based on rules:
        - Rule 1: If current room is dirty, clean it.
        - Rule 2: If current room is clean, move to the other room.
        """
        percept = self.perceive(env)

        if percept == 'dirty':
            clean_room(env, self.location)
            return "Cleaned the room"
        else:
            self.move()
            return "Moved to the other room"

    def move(self):
        """Switches the agent's location between Room A and Room B."""
        if self.location == 'A':
```

```
            self.location = 'B'
        else:
            self.location = 'A'
```

Task 3: Run the Simulation:

```
def run_simulation():
    # Initialize environment
    env = initialize_environment()

    # Get user inputs for starting position and simulation steps
    start_loc = input("Enter agent's starting location (A or B): ").strip().upper()
    while start_loc not in ['A', 'B']:
        start_loc = input("Invalid location. Enter starting location (A or B): ").strip().upper()

    steps = int(input("Enter number of steps the simulation should run: "))

    # Instantiate agent
    agent = VacuumAgent(start_location=start_loc)

    print(f"\nInitial Environment State: {env} | Agent Location: Room {agent.location}")
    print("=" * 55)

    # Run simulation loop
    for step in range(1, steps + 1):
        loc_before_action = agent.location
        action_taken = agent.act(env)

        print(f"Step {step}:")
        print(f"  - Agent's Current Location : Room {loc_before_action}")
        print(f"  - Action Taken             : {action_taken}")
        print(f"  - Current Environment State: {env}")
        print("-" * 55)

if __name__ == "__main__":
    run_simulation()
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. Rule 1: If the current room is dirty, clean it.
   Rule 2: If the current room is clean, move to the other room.

2. The agent checks its percept by looking at the state of its current room in the environment dictionary. If the state evaluates to 'dirty', it executes the cleaning rule; if it evaluates to 'clean', it executes the movement rule to switch rooms
3. If both rooms are clean from the beginning, the agent will continuously alternate (oscillate) back and forth between Room A and Room B at every time step without performing any cleaning actions, because perceiving a clean room triggers Rule 2 (moving to the other room).
4. To support more than two rooms (e.g., Rooms A, B, and C), the environment representation would be expanded, and the agent's `move()` method would be updated to navigate through a graph structure, queue, or random selection algorithm (as explored in the bonus task) to ensure every dirty room is systematically visited and cleaned.